

Lauren Chisholm
CS 5001
Final Project - Report
4/19/2026

1) Project Description

This project creates a WAV file from a .txt file. It uses File I/O to open and read an outside score, parses and splits information into lists, uses dictionaries to translate musical notation in a custom DSL (A4Q) to a readable format (wave frequency = 440 hz, samples = 22050, BPM = 120), that is then written to a 16-bit WAV audio format. There is a comprehensive unit testing suite, covering correct and incorrect test results, and a simple linter for the .txt files to ensure that there are 4 beats per measure for each file.

2) Changes from Planning Document 2

Looking at my planning document 2, I'm noticing a number of changes:

1) Use of numpy: My initial plan was to use NumPy to calculate the frequency per sample. Since I ended up working with square waves, it turned out that a simple list was all I needed.

2) Use of recursive structure: I was thinking of using recursion in the Song class to assemble the actual song waveform before packaging into WAV format. As I worked on it, I realized that for loops and appending accomplished the same thing.

3) Addition of validation/linter functionality: I realized partially through building the score.txt files that I was building a rudimentary DSL (domain specific language). I quickly realized that while it was mostly manageable to count beats in a 8-9 line melody, it'd be incredibly difficult to do so in a longer file. As a result, I built a secondary linter that defines a class that checks a song bar by bar to see how long the bar itself is. If there are too few or too many notes in a bar of music for its time signature, it will get out of sync with itself and sound off. I wanted to avoid that, so I built score_linter.py to check for errors in time signatures before the file was parsed and read to a WAV file.

4) Two channel structure in 'Alicia': Ultimately, transposing the 6/8 original to a two-channel structure in 4/4 time proved to be too challenging for me. While I'd wanted to have a simple bass line, I wasn't able to build one that both syncopated with the song and was in 4/4 time. I ended up simply transcribing the melody alone.

5) Parsing approach: My initial plan was to parse the individual note_string using a combination of len(note) and index mapping. So everything past note[1] would be duration, and note[0,1] would be pitch. However, as I thought about it more, there were too many exceptions, and since notes can be anywhere from 3 characters (A4C) to 5 characters (A#4DQ), the 'else' statements quickly started piling up. I realized I could set the number in each note_string as a sort of checkpoint; there will only ever be one number in a note, and it will always be the last character in self.pitch. It just made sense to retool the logic into: "iterate through this string. If you haven't seen the number yet, add the thing you're looking at to string 'pitch'. Once you see the number, add that

number to 'pitch'. Everything afterward gets added to 'duration'. It's flexible because it doesn't rely on indices.

3) Reflection

1) I learned a lot in this project - namely practical debugging in real time, DSL creation, file writing, implementing libraries (and when not to implement libraries). I also took this opportunity to indulge in my desire to be defensive in planning for input errors; we didn't get as much of a chance to do that in class as I'd have liked. I enjoyed thinking about the most obvious weak points and protecting against them. Overall I think this was a great opportunity to practice and get used to managing errors, fixing and tinkering and tracing issues through code.

I spent a LOT of time early on trying to get the file to output to the correct location. I knew hardcoding a file path was a bad move, but I used it initially to test that the WAV files themselves were working. From there, I looked for a responsive and dynamic method for file paths - essentially a 'whatever folder this .py file is in, look there and nowhere else. And write there and nowhere else.' I'm sure my computer is littered with 'cscale_output.wav' files in all sorts of places. I found one in my downloads folder somehow. Luckily I found the python documentation for `os.path.join`, which worked the first time and all the times afterward.

2) If I could start over on this project, I think I'd be more thoughtful about song choice. I chose 'Alicia' from Clair Obscur: Expedition 33 because I was playing the DLC around the time the project was announced. I overestimated my musical theory knowledge and thought that transcribing a treble and bass line into 4/4 from 6/8 would be relatively simple. It was not. I ended up only transcribing a treble line, and I wish I'd had the foresight to pick another song. I really wanted two channels for all the songs I created. I could have chosen a second song ("[Lumiere](#)") from Clair Obscur: Expedition 33 that's in 4/4 time; I even considered it. However, I decided on 'Alicia' because the song was prettier. It is, but I wish I'd have gone with the 4/4 song.

On the more CS side of this project, there's one deliberate thing I'd like to change, but I don't know how and I really don't want to mess with my code at this point. I repeat tons of code in `load_score()`. The bass channel and treble channel parsing sections are identical and if I were doing this over, I'd definitely create a single function like `'parse_channel(channel, notes_list)'` and call it twice inside `load_score`. That'd be FAR neater and less redundant.

4) Acknowledgments/Citations

Program Design:

1) How To Make a Simple Sound Synthesizer in Python
<https://www.youtube.com/watch?v=ITk2QMjCMao&list=WL&index=34>

2) Code a Wavetable Synth with Python in 6 Minutes Tutorial
<https://www.youtube.com/watch?v=zBFET8fkjfl&list=WL&index=35>

3) How to Build a Synthesizer in Python!

<https://www.youtube.com/watch?v=InGrKBRRUCuc&list=WL&index=31>

4) Code-It-Yourself! Sound Synthesizer #1 - Basic Noises

<https://www.youtube.com/watch?v=tgamhuQnOkM>

Functions:

<https://www.geeksforgeeks.org/python/zip-in-python/>

<https://docs.python.org/3/library/unittest.html#unittest.TestCase.assertTrue>

Libraries:

<https://docs.python.org/3/library/wave.html>

<https://docs.python.org/3/library/os.path.html>

<https://docs.python.org/3/library/struct.html>

Musical Theory/Electronic Music Generation:

1) How Oldschool Sound/Music worked

https://www.youtube.com/watch?v=q_3d1x2VPxk&list=WL&index=32

2) What is One-Bit Sound? Square waves & Pulse Waves

<https://www.youtube.com/watch?v=iOXTXut00yo&list=WL&index=34>

3) The Math Behind Music and Sound Synthesis

<https://www.youtube.com/watch?v=Y7TesKMSE74>

4) Making WAVs: Understanding, Parsing, and Creating WAV Files in JavaScript

<https://www.youtube.com/watch?v=udbA7u1zYfc&t=1s>

Sheet Music/Recordings:

1) 'Alicia' from Clair Obscur: Expedition 33(2025). Composer: Lorien Testard

Sheet Music: <https://www.scribd.com/document/893703884/Alicia>

Recording Reference(s):

<https://www.youtube.com/watch?v=TBl-7SB8zIY>

https://www.youtube.com/watch?v=05Wv4hQZA_0&list=RD05Wv4hQZA_0&start_radio=1

2) 'Doctor Who Theme' from Doctor Who(1963). Composers: Ron Grainer and Delia Darbyshire

Sheet Music: <https://www.scribd.com/document/503022712/Doctor-Who-Theme-easy>

Recording Reference:

https://www.youtube.com/watch?v=8eqrADR26no&list=RD8eqrADR26no&start_radio=1

3) 'All Star' by Smash Mouth(1999). Composer: Greg Camp

Sheet Music: <https://musescore.com/user/27245531/scores/4862161?srsltid=AfmBOoo-11MVVDRI2dkIKMmLwZuquhk1WZBo6rxrLRycxliZhdKwPa1O>

Recording Reference:

https://www.youtube.com/watch?v=L_jWHffIx5E&list=RDL_jWHffIx5E&start_radio=1

Rubber Duck Services:

I used Claude(Anthropic) as a rubber duck during coding, a second pair of eyes on syntax during debugging, and a resource for finding sources on signal processing calculation. All prompts are available upon request.